

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Северо-Кавказский федеральный университет»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

КУРСОВАЯ РАБОТА

по дисциплине

«ИНТЕРНЕТ ТЕХНОЛОГИИ»

на тему:

«Разработка веб-приложения для записи пациентов на приём»

Выполнил:

Гадиян Сергей Гариевич

Студент 4 курса группы ПИН-б-з-22-1

Направления подготовки

09.03.03 Прикладная информатика

заочной формы обучения

(подпись)

Руководитель работы:

(ФИО, должность)

Работа допущена к защите _____
(подпись руководителя) (дата)

Работы выполнена и
защищена с оценкой _____ Дата защиты _____

Члены комиссии: _____
(должность) (подпись) (И.О. Фамилия)

Ставрополь, 2026 г.

УТВЕРЖДАЮ

Директор департамента цифровых,
робототехнических систем
и электроники
Азаров И.В.

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники
Направление подготовки 09.03.03 Прикладная информатика
Направленность (профиль) Прикладная информатика в экономике

ЗАДАНИЕ
на курсовую работу/проект

студента Гадиян Сергея Гариевича

по дисциплине Интернет-технологии

1. Тема работы Разработка веб-приложения для записи пациентов на приём
2. Цель Систематизация теоретических знаний в области интернет-технологий и их практическое применение для создания веб-приложения, направленного на решение актуальной задачи в выбранной предметной области.
3. Задачи В ходе выполнения курсовой работы предполагается провести анализ архитектурных принципов построения веб-приложений, исследовать подходы к аутентификации и авторизации пользователей, обосновать выбор инструментальных средств разработки и развёртывания, а также реализовать практическую часть проекта с применением современных методов контейнеризации, облачного деплоя и систем контроля версий
4. Перечень подлежащих разработке вопросов:
 - а) по теоретической части Архитектура клиент-серверных веб-приложений, принципы работы веб-сервера Nginx и его роль как обратного прокси, разработка серверной логики на платформе Node.js с использованием фреймворка Express, проектирование реляционных баз данных на MySQL 8.0, контейнеризация приложений с помощью Docker, методы аутентификации и авторизации пользователей в веб-приложениях, а также развёртывание готовых решений на облачных платформах.
 - б) по практической части Постановка задачи и выбор средств реализации (HTML, CSS, JavaScript, Node.js, Express, MySQL, Nginx, Docker), проектирование объектной модели и базы данных, разработка клиентской и серверной частей, вёрстка пользовательского интерфейса, тестирование работоспособности, контейнеризация и развёртывание на облачной платформе.

5. Исходные данные:

- а) по литературным источникам

б) по вариантам, разработанным преподавателем

Разработка веб-приложения для записи пациентов на приём

в) иное _____

6. Список рекомендуемой литературы Голован, С.В. REST API: проектирование и разработка. — Москва: Альпина Паблишер, 2023. — 248 с.

Nginx Documentation [Электронный ресурс]. URL: <https://nginx.org/ru/docs/>

Docker Documentation [Электронный ресурс]. URL: <https://docs.docker.com/>

Чернышов, Ю.Н. Разработка баз данных в SQL. — Москва: Луч, 2023. — 384

с.

7. Контрольные сроки представления отдельных разделов курсовой работы:

25 % - _____ “____” _____ 20__ г.

50 % - _____ “____” _____ 20__ г.

75 % - _____ “____” _____ 20__ г.

100 % - _____ “____” _____ 20__ г.

8. Срок защиты студентом курсовой работы “____” _____ 20__ г.

Дата выдачи задания “____” _____ 20__ г.

Руководитель курсовой работы

(ученая степень, звание) (личная подпись) (инициалы, фамилия)

Задание принял(а) к исполнению студент _____ формы обучения
_____ курса _____ группы _____

(личная подпись) / _____
(инициалы, фамилия)

Отзыв

на курсовой проект студента 4 курса

Гадиян Сергея Гариевича

тема: «Разработка веб-приложения для записи пациентов на приём»

Актуальность: курсовая работа посвящена разработке веб-приложения для записи пациентов на приём к врачам поликлиник, что является актуальной задачей в условиях цифровизации здравоохранения и роста потребности в удобных электронных сервисах для пациентов.

В первой главе рассмотрены теоретические основы веб-разработки: архитектура клиент-серверных приложений, веб-сервер Nginx, платформа Node.js с фреймворком Express, проектирование баз данных на основе MySQL, технология контейнеризации Docker, методы аутентификации и авторизации, а также развёртывание приложений на облачных платформах.

Вторая глава содержит практическую реализацию: постановку задачи, обоснование выбора технологий, проектирование архитектуры и базы данных, разработку серверной и клиентской частей, контейнеризацию, развёртывание на облачном сервере и тестирование разработанного приложения.

Выводы, сделанные в Заключение, соответствуют целям, поставленным во Введении.

Проанализирован достаточный объём литературы, включающий нормативные документы, учебные издания и техническую документацию.

За время работы студент проявил себя как инициативный и целеустремлённый студент, освоивший современные технологии веб-разработки

Таким образом, работа выполнена на _____ уровне, соответствует требованиям, предъявляемым к курсовым проектам, и заслуживает оценки _____.

Научный руководитель

(степень, звание, должность, место работы, Ф. И. О.)

Печать «____» _____ 20____ г.

Оглавление

ВВЕДЕНИЕ	7
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	10
1.1 Архитектура клиент-серверных веб-приложений.....	10
1.2 Веб-сервер Nginx и его роль как обратного прокси	11
1.3 Серверная разработка на платформе Node.js и фреймворке Express ..	12
1.4 Проектирование реляционных баз данных на основе СУБД MySQL .	14
1.5 Контейнеризация приложений с помощью Docker	15
1.6 Методы аутентификации и авторизации пользователей	17
1.7 Развёртывание веб-приложений на облачных платформах.....	18
2 ПРАКТИЧЕСКАЯ ЧАСТЬ	20
2.1 Постановка задачи и системные требования	20
2.2 Обоснование выбора стека технологий	21
2.3 Проектирование архитектуры и базы данных	23
2.4 Разработка серверной части.....	26
2.5 Разработка клиентской части и вёрстка.....	28
2.6 Автоматизация процессов разработки и развёртывания	32
2.7 Тестирование и верификация работоспособности	34
ЗАКЛЮЧЕНИЕ	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	39

ВВЕДЕНИЕ

Современные информационные технологии прочно вошли во все сферы деятельности человека, включая здравоохранение. Цифровизация медицинских учреждений направлена на повышение качества и доступности медицинских услуг, сокращение очередей и упрощение взаимодействия пациентов с поликлиниками. Одним из ключевых направлений такой цифровизации является внедрение электронных сервисов записи на приём к врачу, позволяющих пациенту самостоятельно выбрать специалиста, удобные дату и время посещения без необходимости личного обращения в регистратуру или продолжительного ожидания на телефонной линии.

Традиционные способы записи на приём – личное обращение в регистратуру или запись по телефону, обладают рядом существенных недостатков: ограниченность рабочими часами учреждения, загруженность телефонных линий, вероятность ошибок при ручном ведении расписания, а также отсутствие у пациента наглядной информации о доступных специалистах и свободном времени. Веб-приложение для записи на приём лишено указанных недостатков: оно обеспечивает круглосуточный доступ к сервису, автоматический учёт занятых и свободных интервалов, а также разграничение прав доступа для различных категорий пользователей. В связи с этим разработка подобного веб-приложения является актуальной задачей.

Объектом исследования является процесс разработки клиент-серверных веб-приложений с применением современных интернет-технологий.

Предметом исследования выступают методы и средства создания веб-приложения для записи пациентов на приём, включая проектирование реляционной базы данных, реализацию серверного программного интерфейса (REST API), разработку клиентского интерфейса, контейнеризацию и развёртывание приложения на облачной платформе.

Целью курсовой работы является систематизация теоретических знаний в области интернет-технологий и их практическое применение для разработки

веб-приложения, предназначенного для записи пациентов на приём к врачам поликлиник.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ архитектурных принципов построения клиент-серверных веб-приложений;
- изучить роль веб-сервера Nginx как обратного прокси-сервера;
- рассмотреть особенности серверной разработки на платформе Node.js с использованием фреймворка Express;
- изучить принципы проектирования реляционных баз данных на основе СУБД MySQL;
- исследовать технологию контейнеризации приложений с помощью Docker;
- рассмотреть методы аутентификации и авторизации пользователей в веб-приложениях;
- изучить подходы к развёртыванию веб-приложений на облачных платформах;
- спроектировать и реализовать веб-приложение для записи пациентов на приём;
- провести тестирование разработанного приложения и выполнить его развёртывание на облачном сервере.

Методами исследования являются анализ научно-технической литературы и документации по веб-технологиям, сравнение и обоснование выбора инструментальных средств разработки, а также практическое моделирование и программная реализация веб-приложения с последующим тестированием.

Практическая значимость работы заключается в том, что разработанное веб-приложение может быть использовано в качестве основы для организации электронной записи пациентов в медицинских учреждениях, а также в качестве учебного примера построения клиент-серверных веб-приложений с

применением современного технологического стека и средств контейнеризации.

Курсовая работа состоит из введения, теоретической и практической частей, заключения, списка использованных источников и приложений. В теоретической части рассматриваются архитектурные принципы построения веб-приложений, применяемые технологии и инструментальные средства. В практической части представлены постановка задачи, обоснование выбора стека технологий, проектирование архитектуры и базы данных, разработка серверной и клиентской частей, автоматизация развёртывания и тестирование приложения.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Архитектура клиент-серверных веб-приложений

Современные веб-приложения в подавляющем большинстве случаев строятся на основе архитектуры «клиент-сервер». Данная архитектура предполагает разделение системы на две взаимодействующие стороны: клиента, который запрашивает ресурсы и услуги, и сервера, который эти ресурсы предоставляет. Такое разделение позволяет распределить вычислительную нагрузку, обеспечить централизованное хранение и обработку данных, а также упростить сопровождение и масштабирование системы [1].

Клиентом в веб-приложении выступает браузер пользователя, который отображает интерфейс и отправляет запросы на сервер. Серверная сторона принимает эти запросы, выполняет необходимую обработку (проверку прав доступа, обращение к базе данных, выполнение бизнес-логики) и возвращает клиенту результат. Обмен данными между клиентом и сервером осуществляется по протоколу HTTP (HyperText Transfer Protocol) либо его защищённой версии HTTPS, использующей шифрование на основе протоколов SSL/TLS [2].

Взаимодействие в архитектуре «клиент-сервер» строится по принципу «запрос - ответ». Клиент формирует HTTP-запрос, содержащий метод (GET, POST, PATCH, DELETE и другие), адрес ресурса (URL), заголовки и, при необходимости, тело запроса. Сервер обрабатывает запрос и формирует HTTP-ответ, включающий код состояния (например, 200 - успешно, 404 - ресурс не найден, 401 - требуется авторизация), заголовки и тело ответа с запрошенными данными. Важной особенностью протокола HTTP является его «безсостоятельность» (statelessness): каждый запрос обрабатывается независимо, а сведения о состоянии сессии должны передаваться явно - например, через заголовки, cookie или токены [2].

В зависимости от распределения функций между клиентом и сервером выделяют несколько разновидностей архитектуры. В классической многостраничной модели сервер при каждом запросе формирует готовую HTML-страницу целиком и отправляет её браузеру. В современной модели одностраничного приложения (Single Page Application, SPA) браузер загружает интерфейс один раз, а в дальнейшем обменивается с сервером только данными (как правило, в формате JSON), динамически обновляя содержимое страницы средствами JavaScript. Второй подход обеспечивает более высокую отзывчивость интерфейса и снижает объём передаваемых данных, в связи с чем он широко применяется при разработке интерактивных веб-приложений [3].

Отдельное распространение получила многоуровневая (многослойная) архитектура, при которой серверная часть, в свою очередь, разделяется на несколько логических слоёв. Наиболее распространённой является трёхуровневая архитектура, включающая слой представления (пользовательский интерфейс), слой бизнес-логики (обработка данных и правил предметной области) и слой данных (хранение информации в базе данных). Разделение на слои повышает модульность системы, упрощает её тестирование и позволяет независимо развивать и заменять отдельные компоненты, не затрагивая остальные [1].

1.2 Веб-сервер Nginx и его роль как обратного прокси

Nginx — это высокопроизводительный веб-сервер, который также применяется как обратный прокси-сервер, балансировщик нагрузки и кэширующий сервер. Он был разработан как решение проблемы обслуживания большого числа одновременных соединений и на сегодняшний день является одним из наиболее распространённых веб-серверов в мире [4].

Ключевой особенностью Nginx является событийно-ориентированная (асинхронная) архитектура обработки запросов. В отличие от серверов, создающих отдельный процесс или поток на каждое соединение, Nginx

обрабатывает множество соединений в рамках ограниченного числа рабочих процессов. Благодаря этому достигается высокая производительность при низком потреблении оперативной памяти, что особенно важно при большой нагрузке [4].

Одной из основных областей применения Nginx является отдача статических файлов - HTML-документов, таблиц стилей CSS, сценариев JavaScript и изображений. С этой задачей Nginx справляется быстро и эффективно, что позволяет разгрузить серверную часть приложения.

Важнейшей функцией Nginx является работа в режиме обратного прокси-сервера. Обратный прокси принимает запросы от клиентов и перенаправляет их внутренним серверам приложений, а полученный ответ возвращает клиенту. Применение обратного прокси даёт ряд преимуществ: сокрытие внутренней архитектуры приложения, централизованную обработку запросов, возможность терминирования защищённых соединений (SSL/TLS), а также распределение нагрузки между несколькими экземплярами приложения [5].

Типичная схема использования Nginx в современном веб-приложении предполагает следующее разделение обязанностей. Запросы к статическим ресурсам Nginx обрабатывает самостоятельно, а запросы к программному интерфейсу (API) перенаправляет на сервер приложения, работающий на отдельном порту. Такой подход объединяет преимущества быстрой отдачи статики и гибкости серверной логики, обеспечивая при этом единую точку доступа к приложению [5].

1.3 Серверная разработка на платформе Node.js и фреймворке Express

Node.js представляет собой программную платформу, позволяющую выполнять код на языке JavaScript вне браузера - на стороне сервера. Платформа построена на движке V8, разработанном компанией Google для браузера Chrome, что обеспечивает высокую скорость исполнения программ.

Появление Node.js дало возможность использовать единый язык программирования как для клиентской, так и для серверной части веб-приложения, что упрощает разработку и сопровождение проектов [6].

Отличительной чертой Node.js является неблокирующая модель ввода-вывода, основанная на событиях. Вместо того чтобы ожидать завершения каждой операции (например, чтения из базы данных или файла), платформа продолжает обработку других запросов, а результат операции обрабатывается по мере готовности через механизм обратных вызовов и асинхронных функций. Такая модель позволяет одному серверному процессу эффективно обслуживать большое число одновременных подключений, что особенно важно для веб-приложений с интенсивным сетевым взаимодействием [6].

Важной составляющей экосистемы Node.js является менеджер пакетов npm (Node Package Manager), предоставляющий доступ к обширному репозиторию готовых библиотек. С его помощью разработчик может подключать к проекту необходимые модули, а также управлять зависимостями приложения через файл package.json, в котором фиксируются используемые библиотеки и их версии [6].

Для упрощения разработки серверной части на платформе Node.js широко применяется фреймворк Express. Express представляет собой минималистичный и гибкий веб-фреймворк, предоставляющий набор средств для обработки HTTP-запросов, маршрутизации и формирования ответов. Он не навязывает разработчику жёсткую структуру приложения, что делает его удобным инструментом для создания как небольших сервисов, так и полноценных программных интерфейсов [7].

Центральными понятиями Express являются маршрутизация и промежуточное программное обеспечение (middleware). Маршрутизация определяет, какой обработчик будет вызван в ответ на запрос с конкретным методом и адресом. Промежуточное программное обеспечение представляет собой функции, которые последовательно обрабатывают запрос до того, как он достигнет основного обработчика: с их помощью реализуются разбор тела

запроса, проверка прав доступа, логирование и обработка ошибок. Такая организация позволяет строить серверную логику из независимых, последовательно выполняемых компонентов [7].

Сочетание Node.js и Express является одним из наиболее распространённых решений для построения серверной части веб-приложений и реализации архитектурного стиля REST. Оно обеспечивает высокую производительность, простоту разработки и доступ к большому количеству готовых библиотек, что делает данный стек технологий удобным выбором для создания современных веб-сервисов [7].

1.4 Проектирование реляционных баз данных на основе СУБД MySQL

Большинство веб-приложений нуждается в надёжном хранении данных, которое обеспечивается системами управления базами данных (СУБД). Наиболее распространённым типом являются реляционные СУБД, в которых данные представлены в виде совокупности связанных между собой таблиц. Каждая таблица состоит из строк (записей) и столбцов (полей), а взаимодействие с данными осуществляется с помощью языка структурированных запросов SQL (Structured Query Language) [8].

MySQL представляет собой одну из наиболее популярных реляционных систем управления базами данных. Она отличается высокой производительностью, надёжностью, открытостью исходного кода и широкой поддержкой со стороны сообщества разработчиков. Благодаря этим качествам MySQL широко применяется при создании веб-приложений различного масштаба - от небольших сайтов до крупных информационных систем [8].

Реляционная модель данных опирается на ряд базовых понятий. Первичный ключ (primary key) - это поле или совокупность полей, однозначно идентифицирующих каждую запись таблицы. Внешний ключ (foreign key) - это поле, ссылающееся на первичный ключ другой таблицы, что позволяет устанавливать связи между таблицами и поддерживать ссылочную

целостность данных. Применение внешних ключей предотвращает появление записей, ссылающихся на несуществующие данные, и обеспечивает согласованность информации в базе [8].

Между таблицами реляционной базы данных могут устанавливаться связи различных типов: «один к одному», «один ко многим» и «многие ко многим». Наиболее распространённой является связь «один ко многим», при которой одной записи в первой таблице соответствует несколько записей во второй. Например, одной поликлинике может соответствовать множество врачей, а одному врачу - множество записей пациентов на приём. Правильное определение связей между таблицами является важнейшим этапом проектирования базы данных [8].

Важным элементом проектирования реляционной базы данных является нормализация - процесс организации данных, направленный на устранение избыточности и предотвращение аномалий при добавлении, изменении и удалении записей. Нормализация предполагает разделение данных на отдельные таблицы таким образом, чтобы каждый факт хранился в единственном месте. Это снижает вероятность возникновения противоречий и упрощает сопровождение базы данных [8].

Для наглядного представления структуры базы данных применяются диаграммы «сущность - связь» (Entity-Relationship Diagram, ER-диаграмма). На такой диаграмме отображаются сущности (таблицы), их атрибуты (поля) и связи между ними. ER-диаграмма позволяет на этапе проектирования получить целостное представление о структуре данных, выявить необходимые связи и избежать ошибок до начала реализации приложения [8].

1.5 Контейнеризация приложений с помощью Docker

При разработке и развёртывании веб-приложений нередко возникает проблема различий между средами, в которых выполняется программа. Приложение, успешно работающее на компьютере разработчика, может вести себя иначе на сервере из-за отличий в версиях операционной системы,

установленных библиотек и настроек окружения. Для решения этой проблемы применяется технология контейнеризации, наиболее известным средством которой является Docker [9].

Контейнеризация представляет собой способ упаковки приложения вместе со всеми его зависимостями - библиотеками, настройками и средой выполнения - в изолированный программный модуль, называемый контейнером. Контейнер содержит всё необходимое для запуска приложения и работает одинаково в любой среде, где установлен Docker. Это обеспечивает переносимость приложения и устраняет проблему несовместимости окружений [9].

Контейнеры следует отличать от виртуальных машин. Виртуальная машина эмулирует аппаратное обеспечение и содержит полноценную гостевую операционную систему, что требует значительных ресурсов. Контейнеры же используют ядро операционной системы хоста и изолируют лишь процессы и их зависимости, благодаря чему они занимают меньше места, быстрее запускаются и потребляют меньше памяти. Это делает контейнеризацию эффективным средством развёртывания приложений [9].

Основой создания контейнера является образ (image) - неизменяемый шаблон, содержащий файловую систему и инструкции для запуска приложения. Образ создаётся на основе файла Dockerfile, в котором описывается последовательность команд по сборке: выбор базового образа, копирование файлов приложения, установка зависимостей и определение команды запуска. Из готового образа создаётся запущенный экземпляр - контейнер [9].

Современные веб-приложения, как правило, состоят из нескольких взаимодействующих компонентов - веб-сервера, сервера приложения и базы данных. Для совместного управления такими компонентами применяется инструмент Docker Compose. Он позволяет описать все контейнеры приложения, их параметры, зависимости и связи между ними в едином конфигурационном файле формата YAML, после чего вся группа контейнеров

запускается одной командой. Это значительно упрощает развёртывание многокомпонентных приложений и обеспечивает воспроизводимость окружения [9].

1.6 Методы аутентификации и авторизации пользователей

В веб-приложениях, работающих с персональными данными нескольких категорий пользователей, важнейшее значение имеют процессы аутентификации и авторизации. Несмотря на близость этих понятий, они решают разные задачи. Аутентификация - это процесс проверки подлинности пользователя, то есть установление того, что пользователь является тем, за кого себя выдаёт. Авторизация - это процесс определения прав доступа уже аутентифицированного пользователя, то есть установление того, какие действия ему разрешено выполнять [10].

Наиболее распространённым способом аутентификации является проверка пары «логин - пароль». Пользователь вводит свои учётные данные, которые сравниваются с данными, хранящимися на сервере. При успешном совпадении система признаёт пользователя подлинным и предоставляет ему доступ. Для обеспечения безопасности пароли не должны храниться в базе данных в открытом виде: применяется их хеширование - необратимое преобразование пароля в строку фиксированной длины с помощью специальных криптографических алгоритмов. При проверке сравниваются не сами пароли, а их хеши, что защищает учётные данные даже в случае несанкционированного доступа к базе [10].

Поскольку протокол HTTP не сохраняет состояние между запросами, после успешной аутентификации необходимо каким-либо образом «запоминать» вошедшего пользователя. Для этого применяются механизмы управления сессиями. В классическом подходе сервер создаёт сессию и сохраняет сведения о ней, а клиенту передаётся идентификатор сессии, который хранится в cookie и отправляется при каждом последующем запросе [10].

Альтернативным и широко распространённым подходом является использование токенов, в частности технологии JWT (JSON Web Token). Токен представляет собой подписанную сервером строку, содержащую сведения о пользователе и его правах. После аутентификации сервер выдаёт клиенту токен, который тот предъявляет при каждом запросе, как правило, в заголовке. Сервер проверяет подлинность подписи токена и на основании его содержимого определяет пользователя и его права. Преимуществом такого подхода является отсутствие необходимости хранить сведения о сессиях на сервере, что упрощает масштабирование приложения [10].

Авторизация чаще всего реализуется на основе ролевой модели доступа (Role-Based Access Control, RBAC). В рамках этой модели каждому пользователю назначается определённая роль, а права доступа определяются не для отдельных пользователей, а для ролей. Такой подход упрощает управление правами: при изменении полномочий достаточно изменить настройки роли, а не каждого пользователя в отдельности. Разграничение доступа по ролям позволяет предоставлять различным категориям пользователей разный набор доступных функций в пределах одного приложения [10].

1.7 Развёртывание веб-приложений на облачных платформах

После завершения разработки веб-приложение необходимо развернуть, то есть разместить на сервере, доступном пользователям через сеть Интернет. В настоящее время для этой цели всё чаще применяются облачные платформы, предоставляющие вычислительные ресурсы в аренду через Интернет. Облачные технологии позволяют отказаться от приобретения и обслуживания собственного физического оборудования, что снижает затраты и повышает гибкость [11].

Принято выделять несколько моделей предоставления облачных услуг. Модель «инфраструктура как услуга» (Infrastructure as a Service, IaaS) предоставляет пользователю виртуальные серверы, на которых он

самостоятельно устанавливает операционную систему и необходимое программное обеспечение. Модель «платформа как услуга» (Platform as a Service, PaaS) предоставляет готовую среду для запуска приложений, избавляя разработчика от управления инфраструктурой. Модель «программное обеспечение как услуга» (Software as a Service, SaaS) предоставляет конечному пользователю готовое приложение через Интернет. При развёртывании собственных веб-приложений наиболее часто используется модель IaaS, обеспечивающая максимальную гибкость настройки [11].

При использовании модели IaaS разработчику предоставляется виртуальная машина – программный аналог физического компьютера, на котором установлена операционная система, чаще всего семейства Linux.

Процесс развёртывания приложения на облачном сервере включает ряд этапов: создание виртуальной машины, установку необходимого программного обеспечения, перенос исходного кода приложения, его настройку и запуск. Применение технологии контейнеризации значительно упрощает этот процесс: вместо ручной установки всех зависимостей на сервере достаточно установить Docker и запустить заранее подготовленные контейнеры. Это обеспечивает идентичность среды разработки и среды развёртывания, снижая вероятность ошибок [11].

Важными аспектами развёртывания являются также вопросы доступа к приложению и безопасности. Для того чтобы приложение стало доступно из сети Интернет, виртуальной машине присваивается публичный IP-адрес, а в настройках сетевого экрана (группы безопасности) открываются необходимые сетевые порты. При этом следует открывать только те порты, которые действительно используются приложением, что снижает риски несанкционированного доступа и повышает общую защищённость системы [11].

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Постановка задачи и системные требования

Целью практической части курсовой работы является разработка веб-приложения для записи пациентов на приём к врачам поликлиник. Приложение должно предоставлять удобный интерфейс для самостоятельной записи пациентов, а также инструменты для работы медицинского персонала и сотрудников регистратуры.

В соответствии с поставленной целью приложение должно реализовывать следующие функциональные возможности:

- регистрация и авторизация пользователей с разграничением прав доступа по ролям;
- ведение справочников городов, поликлиник и врачей;
- самостоятельная запись пациента на приём к врачу выбранной поликлиники с выбором свободных даты и времени;
- автоматическое исключение уже занятых интервалов времени из списка доступных для записи;
- просмотр и отмена пациентом собственных записей;
- просмотр врачом записей пациентов к нему и изменение их статуса («приём состоялся», «неявка»);
- просмотр сотрудником регистратуры всех записей с возможностью поиска и сортировки;
- проверка корректности вводимых данных на стороне клиента и сервера.

В приложении предусмотрены три роли пользователей с различными правами. Пациент имеет возможность регистрироваться, записываться на приём и управлять своими записями. Врач просматривает записи пациентов к себе и отмечает результат приёма. Администратор (сотрудник регистратуры) обладает доступом ко всем записям и средствам их поиска и сортировки. Разграничение прав доступа реализовано на основе ролевой модели.

Помимо функциональных, к приложению предъявляются нефункциональные требования. Интерфейс должен быть интуитивно понятным и корректно отображаться как на персональных компьютерах, так и на мобильных устройствах. Приложение должно обеспечивать целостность и согласованность данных, а также защиту от некорректного ввода. Серверная часть должна устойчиво обрабатывать запросы нескольких пользователей одновременно.

Для функционирования приложения определены требования к программно-аппаратному обеспечению сервера и клиента. Серверной платформой выступает виртуальная машина облачного провайдера со следующими характеристиками: операционная система Ubuntu 22.04 LTS, один виртуальный процессор (vCPU), 2 ГБ оперативной памяти и твердотельный накопитель (SSD) объёмом 20 ГБ. На сервере устанавливается система контейнеризации Docker, обеспечивающая запуск всех компонентов приложения.

Со стороны клиента специальных требований к программному обеспечению не предъявляется: для работы с приложением достаточно любого современного веб-браузера (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) с поддержкой технологий HTML5, CSS3 и JavaScript. Приложение имеет адаптивный интерфейс и поддерживает работу как на настольных компьютерах, так и на мобильных устройствах, что не требует от пользователя установки дополнительных программ.

2.2 Обоснование выбора стека технологий

Выбор технологического стека является одним из ключевых решений при разработке веб-приложения, поскольку он определяет трудоёмкость разработки, производительность и удобство дальнейшего сопровождения. При выборе средств реализации учитывались следующие критерии: доступность и распространённость технологии, наличие документации и обучающих материалов, простота освоения, а также пригодность для решения поставленной задачи.

Для реализации серверной части рассматривались несколько платформ. В таблице 1 приведено сравнение наиболее распространённых вариантов.

Таблица 1 – Сравнение платформ для серверной разработки

Платформа	Язык	Особенности
Node.js + Express	JavaScript	Единый язык с клиентом, большая экосистема, низкий порог входа
Go + Gin	Go	Высокая производительность, но требует освоения отдельного языка
Django	Python	Мощный фреймворк, но избыточен для небольшого проекта
Spring Boot	Java	Развитая платформа, но высокая сложность и громоздкость

В качестве серверной платформы выбрана связка Node.js и фреймворка Express. Основным доводом в пользу данного выбора является возможность использования единого языка программирования (JavaScript) как для серверной, так и для клиентской части, что снижает трудозатраты на изучение нескольких языков и упрощает разработку. Кроме того, Node.js обладает обширной экосистемой готовых библиотек, подробной документацией и большим сообществом, а фреймворк Express отличается простотой и гибкостью. Несмотря на то что платформа Go обеспечивает более высокую производительность, для учебного приложения с умеренной нагрузкой решающее значение имели доступность и скорость разработки.

Для хранения данных рассматривались реляционные и нереляционные системы управления базами данных. Поскольку предметная область приложения имеет чёткую структуру с явными связями между сущностями (поликлиники, врачи, пациенты, записи), выбор сделан в пользу реляционной СУБД. В таблице 2 приведено сравнение рассмотренных вариантов.

Таблица 2 – Сравнение систем управления базами данных

СУБД	Тип	Особенности
MySQL	Реляционная	Распространённость, надёжность, простота, открытый код
PostgreSQL	Реляционная	Богатый функционал, но избыточен для данной задачи
MongoDB	Документная	Гибкая схема, но слабее для строго связанных данных

В качестве системы управления базами данных выбрана MySQL. Она хорошо подходит для хранения структурированных данных со связями, отличается надёжностью и высокой производительностью, имеет открытый исходный код и широко применяется в веб-разработке. Реляционная модель MySQL позволяет естественным образом описать связи между поликлиниками, врачами и записями пациентов, а также обеспечить целостность данных при помощи внешних ключей.

Клиентская часть приложения реализована с использованием базовых веб-технологий - HTML, CSS и JavaScript — без применения дополнительных фреймворков. Для учебного проекта с относительно небольшим числом экранов такой подход является оправданным: он не требует освоения сложных инструментов, обеспечивает полный контроль над кодом и снижает объём внешних зависимостей. При этом достигается необходимая интерактивность интерфейса и его адаптивность для мобильных устройств.

В качестве веб-сервера выбран Nginx, выполняющий роль обратного прокси-сервера: он отдаёт статические файлы клиентской части и перенаправляет запросы к программному интерфейсу на серверную часть приложения. Для развёртывания и управления компонентами приложения применяется технология контейнеризации Docker совместно с инструментом Docker Compose, что обеспечивает единообразие среды и упрощает запуск приложения на сервере. Таким образом, итоговый стек технологий включает Node.js с фреймворком Express, СУБД MySQL, веб-сервер Nginx, а также средства контейнеризации Docker.

2.3 Проектирование архитектуры и базы данных

Разрабатываемое приложение построено по трёхуровневой архитектуре, в которой выделяются слой представления, слой бизнес-логики и слой данных. Такое разделение обеспечивает модульность системы и независимость её компонентов. Каждый уровень реализован в виде отдельного программного компонента и запускается в собственном контейнере Docker.

Слой представления отвечает за взаимодействие с пользователем и реализован в виде клиентской части на основе HTML, CSS и JavaScript. Его обслуживает веб-сервер Nginx, который отдаёт статические файлы браузеру и перенаправляет запросы к программному интерфейсу на серверную часть. Слой бизнес-логики реализован на платформе Node.js с использованием веб-фреймворка Express и отвечает за обработку запросов, проверку прав доступа и выполнение правил предметной области. Слой данных представлен системой управления базами данных MySQL, обеспечивающей хранение и согласованность информации.

Взаимодействие компонентов происходит следующим образом: браузер обращается к веб-серверу Nginx, который либо возвращает запрошенные файлы интерфейса, либо передаёт запрос серверу приложения; сервер приложения обрабатывает запрос, при необходимости обращается к базе данных и возвращает результат клиенту. Все три компонента запускаются в отдельных контейнерах и объединяются средствами Docker Compose.

В основе слоя данных лежит реляционная база данных, состоящая из пяти связанных таблиц: cities (города), clinics (поликлиники), doctors (врачи), users (пользователи) и appointments (записи на приём). Структура базы данных и связи между таблицами представлены на рисунке 1.

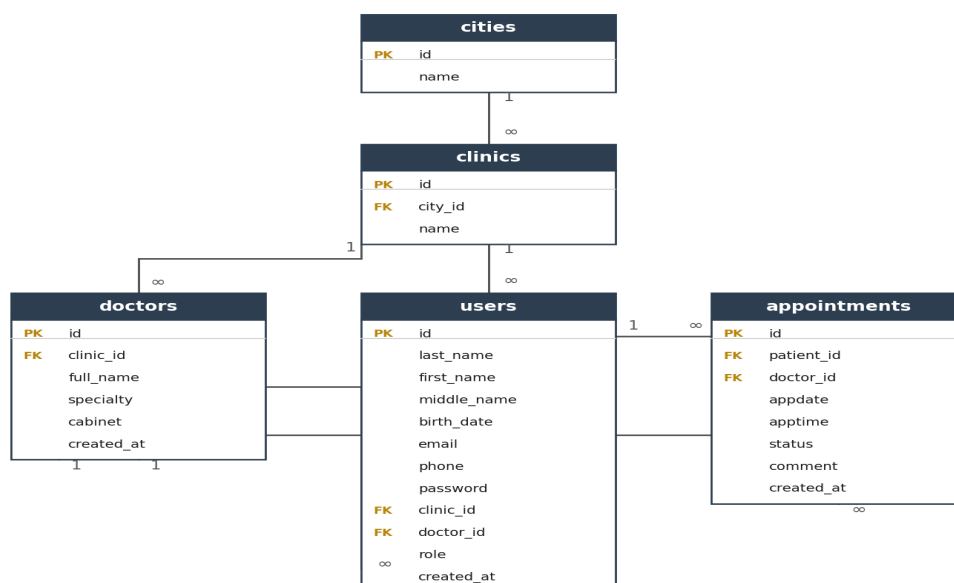


Рисунок 1 – Схема базы данных (ER-диаграмма)

Таблица `cities` содержит перечень городов и состоит из идентификатора и названия. Таблица `clinics` содержит поликлиники, каждая из которых связана с городом через внешний ключ `city_id`. Таблица `doctors` хранит сведения о врачах (ФИО, специальность, кабинет) и связана с поликлиникой через внешний ключ `clinic_id`.

Таблица `users` является центральной и хранит данные всех пользователей системы независимо от их роли. Поле `role` определяет роль пользователя и принимает одно из значений: `patient` (пациент), `doctor` (врач) или `admin` (администратор). Для пользователей с ролью `patient` поле `clinic_id` используется для хранения ссылки на поликлинику, к которой прикреплен пациент. Для пользователей с ролью `doctor` используется поле `doctor_id`, содержащее ссылку на запись врача в таблице `doctors`.

Таблица `appointments` содержит записи пациентов на приём. Каждая запись связана с пациентом (внешний ключ `patient_id`) и врачом (внешний ключ `doctor_id`) и содержит дату приёма (`appdate`), время приёма (`apptime`), комментарий с жалобами пациента, а также статус. Поле `status` принимает значения `booked` (активна), `cancelled` (отменена), `done` (приём состоялся) или `no_show` (неявка). Поле `created_at` фиксирует дату и время создания записи.

Назначение основных полей таблиц приведено в таблице 3.

Таблица 3 – Описание основных таблиц базы данных

Таблица	Назначение	Основные поля
<code>cities</code>	Города	<code>id</code> , <code>name</code>
<code>clinics</code>	Поликлиники	<code>id</code> , <code>city_id</code> , <code>name</code>
<code>doctors</code>	Врачи	<code>id</code> , <code>clinic_id</code> , <code>full_name</code> , <code>specialty</code> , <code>cabinet</code> , <code>created_at</code>
<code>users</code>	Пользователи (все роли)	<code>id</code> , <code>last_name</code> , <code>first_name</code> , <code>middle_name</code> , <code>birth_date</code> , <code>email</code> , <code>phone</code> , <code>password</code> , <code>clinic_id</code> , <code>doctor_id</code> , <code>role</code> , <code>created_at</code>
<code>appointments</code>	Записи на приём	<code>id</code> , <code>patient_id</code> , <code>doctor_id</code> , <code>appdate</code> , <code>apptime</code> , <code>status</code> , <code>comment</code> , <code>created_at</code>

Связи между таблицами реализованы по принципу «один ко многим» с использованием внешних ключей, что обеспечивает ссылочную целостность данных. Одному городу соответствует несколько поликлиник, одной

поликлинике - несколько врачей и пользователей с ролью patient, одному врачу и одному пациенту - несколько записей на приём. Применение внешних ключей исключает появление записей, ссылающихся на несуществующие данные.

2.4 Разработка серверной части

Серверная часть приложения реализована на платформе Node.js с использованием веб-фреймворка Express. Она предоставляет программный интерфейс (REST API), обрабатывающий запросы клиента, выполняющий бизнес-логику и взаимодействующий с базой данных. Серверная часть организована по модульному принципу: исходный код разделён на отдельные каталоги в соответствии с назначением компонентов.

В каталоге controllers расположены контроллеры, содержащие логику обработки запросов; в каталоге routes – описания маршрутов, связывающих адреса запросов с контроллерами; в каталоге middleware – промежуточные обработчики, в частности модуль проверки прав доступа; в каталоге config – настройки подключения к базе данных. Точкой входа в приложение является файл app.js.

В файле app.js выполняется начальная настройка сервера: подключается разбор JSON-тела запросов, настраивается межсайтовое взаимодействие (CORS), задаётся кодировка ответов и подключаются группы маршрутов. Фрагмент конфигурации сервера приведён в листинге 1.

Листинг 1 – Настройка маршрутов в файле app.js

```
const express = require('express');
const app = express();

app.use(express.json());

app.use('/api/auth', authRoutes);
app.use('/api/doctors', doctorRoutes);
app.use('/api/appointments', appointmentRoutes);
app.use('/api', clinicRoutes);

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Сервер запущен на порту ${PORT}`);
});
```

Разграничение прав доступа реализовано с помощью промежуточного обработчика (middleware). После входа клиент передаёт в заголовках запроса идентификатор и роль пользователя, которые обработчик считывает и помещает в объект запроса для дальнейшего использования. Реализация проверки приведена в листинге 2.

Листинг 2 – Промежуточный обработчик проверки пользователя

```
function authRequired(req, res, next) {
  const id = req.headers['x-user-id'];
  const role = req.headers['x-user-role'];
  const doctorId = req.headers['x-doctor-id'];

  if (!id) {
    return res.status(401).json({ error: 'Требуется вход' });
  }
  req.user = {
    id: Number(id),
    role: role || 'patient',
    doctor_id: doctorId ? Number(doctorId) : null
  };
  next();
}
```

Бизнес-логика приложения сосредоточена в контроллерах. Каждый контроллер отвечает за определённую группу операций: аутентификацию, работу с врачами, поликлиниками и записями на приём. Обращение к базе данных осуществляется при помощи библиотеки `mysql2` с использованием параметризованных запросов, что защищает приложение от внедрения SQL-кода. В качестве примера в листинге 3 приведён обработчик, возвращающий список свободных интервалов времени для выбранного врача на заданную дату.

Листинг 3 – Получение свободных интервалов времени

```
async function freeSlots(req, res) {
  const { doctor_id, appdate } = req.query;
  const [busy] = await pool.query(
    `SELECT apptime FROM appointments
     WHERE doctor_id = ? AND appdate = ?
     AND status = 'booked'`,
    [doctor_id, appdate]
  );
  const busyTimes = busy.map(r => (r.apptime||'').slice(0,5));
  const free = ALL_SLOTS.filter(s => !busyTimes.includes(s));
  res.json(free);
}
```

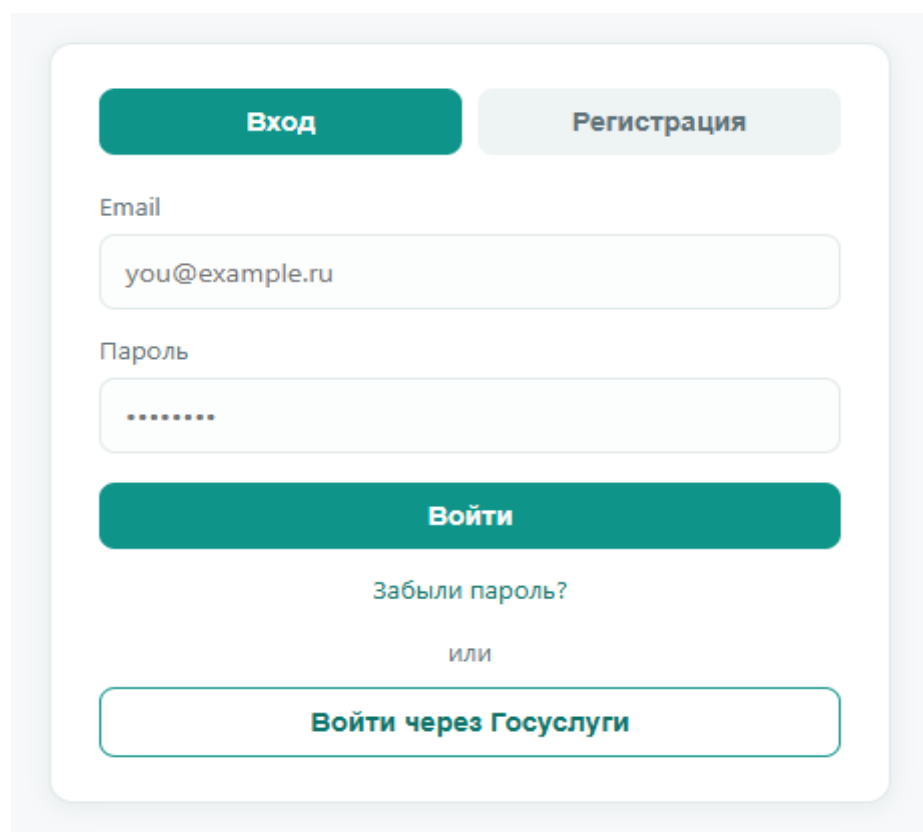
Маршрутизация запросов организована с помощью средств Express: каждому адресу и методу запроса ставится в соответствие функция-обработчик, а для защищённых операций дополнительно указывается промежуточный обработчик проверки доступа. Такой подход обеспечивает чёткое разделение между описанием маршрутов и реализацией бизнес-логики.

2.5 Разработка клиентской части и вёрстка

Клиентская часть приложения отвечает за взаимодействие с пользователем и реализована с использованием технологий HTML, CSS и JavaScript без применения дополнительных фреймворков. Интерфейс построен по принципу одностраничного приложения: после загрузки страницы переключение между экранами и обновление данных происходят без полной перезагрузки, средствами JavaScript. Обмен данными с сервером осуществляется через программный интерфейс с использованием технологии асинхронных запросов (fetch).

Разметка интерфейса описана в файле index.html, оформление - в файле style.css, а логика взаимодействия – в файле app.js. Для оформления применяется адаптивная вёрстка, обеспечивающая корректное отображение интерфейса как на персональных компьютерах, так и на мобильных устройствах. Цветовое решение выдержано в спокойных тонах, характерных для медицинской тематики.

При первом обращении к приложению пользователю отображается экран входа, содержащий форму авторизации с полями для ввода электронной почты и пароля. Внешний вид экрана входа представлен на рисунке 2.



The image shows a login screen for an application. At the top, there are two buttons: a teal button labeled "Вход" (Login) and a light gray button labeled "Регистрация" (Registration). Below these, there is a form with two input fields. The first field is labeled "Email" and contains the text "you@example.ru". The second field is labeled "Пароль" (Password) and contains a series of dots. Below the password field is a teal button labeled "Войти" (Login). Underneath this button, there is a link "Забыли пароль?" (Forgot password?). Below that is the word "или" (or). At the bottom, there is a light gray button with a teal border labeled "Войти через Госуслуги" (Login via Gosuslugi).

Рисунок 2 – Экран входа в приложение

Для создания новой учётной записи пациент переходит на форму регистрации. Форма содержит поля для ввода фамилии, имени, отчества, даты рождения, выбора города и поликлиники, электронной почты, телефона и пароля, а также флажок согласия на обработку персональных данных. При вводе данных выполняется их проверка: контролируются корректность электронной почты, формат телефона, надёжность пароля и возраст пользователя. Внешний вид формы регистрации представлен на рисунке 3.

Вход Регистрация

Фамилия *
Иванов

Имя *
Иван

Отчество
Иванович

Дата рождения *
ДД.ММ.ГГГГ

Город *
г. Ставрополь

Поликлиника *
ГАУЗ СК "Городская поликлиника № 3"

Email *
you@example.ru

Телефон *
+7 () - - -

Пароль *
придумайте пароль

☐ Я даю согласие на обработку персональных данных в соответствии с требованиями №152-ФЗ от 27.07.2006.

* — поля, обязательные для заполнения

Внимание: после регистрации изменить персональные данные можно только при обращении в МФЦ.

Зарегистрироваться

Рисунок 3 – Форма регистрации пациента

После входа пациенту становится доступна форма записи на приём. В ней пользователь выбирает врача из списка специалистов своей поликлиники, указывает дату и время приёма, а также при необходимости вводит комментарий с описанием жалоб. Список доступного времени формируется автоматически: занятые интервалы исключаются, а запись возможна только на будние дни в пределах двух недель. Внешний вид формы записи представлен на рисунке 4.

Записаться на приём

Врач

Выберите врача

Дата

ДД.ММ.ГГГГ

Время

Сначала выберите врача и дату

Жалоба / комментарий (необязательно)

Например: головная боль третий день

Записаться

Мои записи

Записей пока нет.

Рисунок 4 – Форма записи на приём

Записанные на приём пациенты, а также сотрудники медицинского учреждения видят список записей, оформленный в виде таблицы. Состав отображаемой информации зависит от роли пользователя: пациент видит свои записи, врач – записи пациентов к себе с указанием жалоб, администратор – все записи с возможностью поиска и сортировки. Внешний вид списка записей в режиме администратора представлен на рисунке 5.

Все записи пациентов

Поиск по ФИО или телефону

Сначала новые

ДАТА	ВРЕМЯ	ВРАЧ	ПАЦИЕНТ	СИМПТОМЫ	ЗАПИСАН	СТАТУС
30.06.2026	15:00	Сидорова Елена Павловна (Кардиолог)	Гадиян Сергей Гариевич +7 (962) 007-43-40	боль в сердце	22.06.2026 20:18	активна Отменить
26.06.2026	09:30	Кузнецов Дмитрий Игоревич (Невролог)	Иванов Иван Иванович +7 (961) 123-45-77	головная боль	22.06.2026 20:17	активна Отменить

Рисунок 5 – Список записей в режиме администратора

Для повышения удобства использования в интерфейсе применяются модальные окна для подтверждения действий и вывода уведомлений, а также маска ввода номера телефона. Сообщения об ошибках и успешном выполнении операций отображаются непосредственно на странице, что обеспечивает наглядную обратную связь с пользователем.

2.6 Автоматизация процессов разработки и развёртывания

При разработке приложения применялись современные средства автоматизации, обеспечивающие контроль версий исходного кода и упрощённое развёртывание системы. Для управления версиями использовалась система контроля версий Git, а исходный код размещён в удалённом репозитории на платформе GitHub. Применение системы контроля версий позволяет отслеживать историю изменений, возвращаться к предыдущим состояниям проекта и вести разработку в отдельных ветках.

В репозитории проекта используются две основные ветки: ветка `main`, содержащая стабильную версию приложения, и ветка `develop`, предназначенная для разработки. Такой подход отделяет рабочую версию от вносимых изменений и снижает риск нарушения работоспособности проекта. Внешний вид репозитория проекта представлен на рисунке 6.

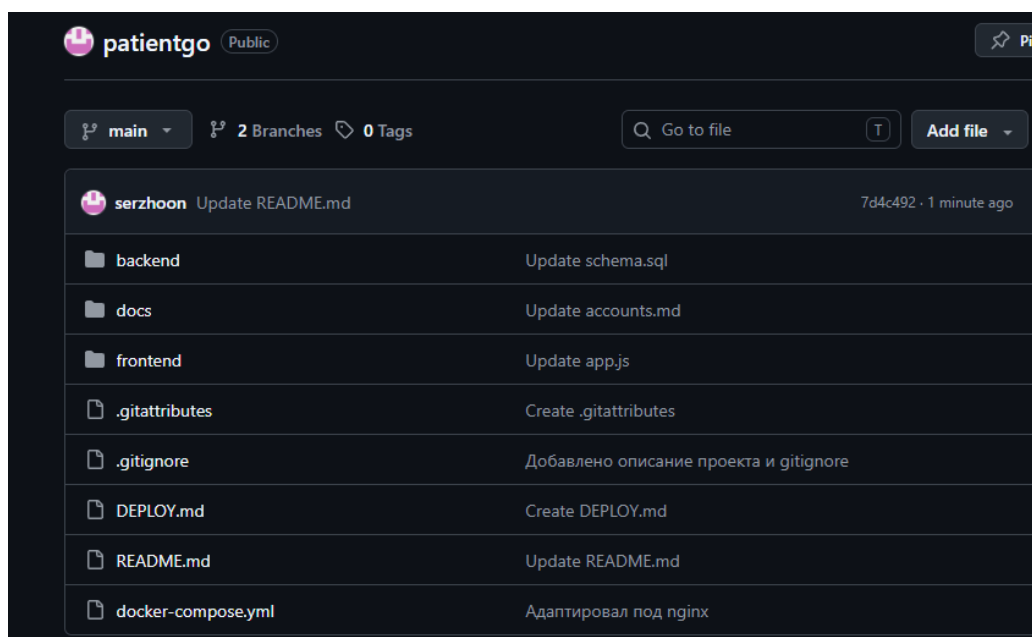


Рисунок 6 – Репозиторий проекта на GitHub

Для развёртывания приложения применяется технология контейнеризации Docker. Каждый компонент системы упакован в отдельный контейнер, а их совместный запуск описан в конфигурационном файле `docker-compose.yml`. Данный файл определяет три сервиса: базу данных MySQL, серверную часть на Node.js и веб-сервер Nginx с клиентской частью. Фрагмент конфигурации с описанием сервиса базы данных приведён в листинге 4.

Листинг 4 – Фрагмент файла `docker-compose.yml`

```
services:
  mysql:
    image: mysql:8.0
    container_name: clinic-mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: clinic
    ports:
      - "3306:3306"
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 5s
```

Серверная часть приложения собирается в образ на основе файла `Dockerfile`, в котором описывается процесс подготовки контейнера: выбор базового образа Node.js, копирование файлов проекта, установка зависимостей и запуск приложения. Содержимое файла `Dockerfile` серверной части приведено в листинге 5.

Листинг 5 – Файл `Dockerfile` серверной части

```
FROM node:18
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "src/app.js"]
```

Параметр `restart: always` в конфигурации контейнеров обеспечивает их автоматический перезапуск при сбое или перезагрузке сервера, что повышает отказоустойчивость приложения. Для корректного порядка запуска

используется механизм зависимостей: серверная часть запускается только после того, как база данных готова к работе.

Процесс развёртывания приложения на облачном сервере состоит из следующих шагов: получение актуальной версии исходного кода из репозитория с помощью команды `git pull` и сборка и запуск контейнеров командой `docker compose up`. Благодаря контейнеризации развёртывание сводится к выполнению нескольких команд, а среда выполнения на сервере полностью соответствует среде разработки. Последовательность команд развёртывания приведена в листинге 6.

Листинг 6 – Команды развёртывания приложения на сервере

```
git pull
sudo docker compose up --build -d
```

Таким образом, применение системы контроля версий Git и технологии контейнеризации Docker позволило автоматизировать процессы разработки и развёртывания, обеспечить воспроизводимость среды выполнения и упростить размещение приложения на облачном сервере.

2.7 Тестирование и верификация работоспособности

После завершения разработки приложения было проведено его тестирование с целью проверки корректности работы всех функциональных возможностей и соответствия системы предъявленным требованиям. Применялось функциональное тестирование методом «чёрного ящика», при котором проверяется поведение системы с точки зрения пользователя без анализа внутреннего устройства программного кода. Тестирование проводилось вручную путём последовательного выполнения типовых пользовательских сценариев.

В ходе тестирования проверялись основные функции приложения для всех ролей пользователей: регистрация и вход, проверка корректности вводимых данных, запись на приём, отмена записи, изменение статуса записи

врачом, а также разграничение прав доступа. Перечень основных тест-кейсов и результаты их выполнения приведены в таблице 4.

Таблица 4 – Результаты функционального тестирования

№	Проверяемая функция	Ожидаемый результат	Итог
1	Регистрация пациента с корректными данными	Учётная запись создана	Успешно
2	Регистрация с некорректным email	Выводится сообщение об ошибке	Успешно
3	Регистрация с возрастом младше 18 лет	Регистрация отклонена	Успешно
4	Вход с верными учётными данными	Выполнен вход в систему	Успешно
5	Вход с неверным паролем	Выводится сообщение об ошибке	Успешно
6	Запись на приём на свободное время	Запись создана	Успешно
7	Исключение занятого времени из списка	Занятое время не отображается	Успешно
8	Отмена записи пациентом	Запись отменена	Успешно
9	Отметка приёма врачом («состоялся»)	Статус записи изменён	Успешно
10	Доступ пациента к чужим записям	Доступ ограничен ролью	Успешно

Отдельное внимание при тестировании уделялось проверке корректности вводимых данных. Приложение контролирует правильность заполнения полей регистрации: формат электронной почты и номера телефона, надёжность пароля, согласие на обработку персональных данных и возраст пользователя. При вводе недопустимых данных пользователю выводится понятное сообщение об ошибке, а отправка формы блокируется. Пример вывода сообщения об ошибке при некорректном вводе представлен на рисунке 7.

Пароль *

☒ Я даю согласие на обработку персональных данных в соответствии с требованиями №152-ФЗ от 27.07.2006.

* — поля, обязательные для заполнения

Внимание: после регистрации изменить персональные данные можно только при обращении в МФЦ.

Зарегистрироваться

Пароль: минимум 8 символов, хотя бы одна буква и одна цифра.

Рисунок 7 – Проверка корректности вводимых данных

Также проверялась корректность работы механизма записи на приём. При выборе врача и даты приложение запрашивает у сервера список свободных интервалов времени, исключая уже занятые. Это предотвращает возможность записи двух пациентов на одно и то же время к одному врачу. Результат успешного создания записи на приём представлен на рисунке 8.

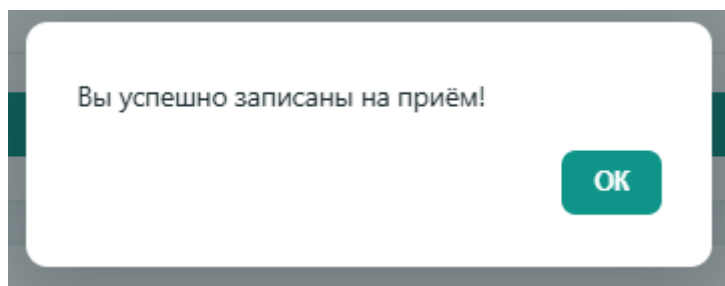


Рисунок 8 – Успешное создание записи на приём

Кроме того, проверялось разграничение прав доступа между ролями. Тестирование подтвердило, что пациент имеет доступ только к собственным записям, врач – к записям пациентов к себе, а администратор – ко всем записям. Попытки доступа к данным, не предусмотренным ролью пользователя, корректно отклоняются серверной частью приложения.

По результатам проведённого тестирования можно сделать вывод, что разработанное приложение функционирует корректно и в полном объёме реализует предъявленные к нему требования.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано веб-приложение для записи пациентов на приём к врачам поликлиник. Поставленная цель достигнута: систематизированы теоретические знания в области интернет-технологий и применены на практике для создания работающего веб-приложения, развёрнутого на облачном сервере.

В процессе работы были решены все поставленные задачи. В теоретической части рассмотрены архитектурные принципы построения клиент-серверных веб-приложений, роль веб-сервера Nginx как обратного прокси, особенности серверной разработки на платформе Node.js с фреймворком Express, принципы проектирования реляционных баз данных на основе СУБД MySQL, технология контейнеризации Docker, методы аутентификации и авторизации, а также подходы к развёртыванию приложений на облачных платформах.

В практической части на основе изученного материала выполнена разработка приложения.

В результате были получены следующие результаты:

- сформулированы функциональные и нефункциональные требования к приложению;
- обоснован выбор технологического стека: Node.js с фреймворком Express, СУБД MySQL, веб-сервер Nginx и средства контейнеризации Docker;
- спроектирована архитектура приложения и реляционная база данных, состоящая из пяти связанных таблиц;
- реализована серверная часть в виде программного интерфейса REST API с разграничением прав доступа по ролям;
- разработана клиентская часть с адаптивным интерфейсом для трёх ролей пользователей;
- выполнена контейнеризация приложения и его развёртывание на облачном сервере;

– проведено функциональное тестирование, подтвердившее корректность работы приложения.

Разработанное приложение реализует три роли пользователей – пациент, врач и администратор (регистратура) – с различными возможностями. Пациент может самостоятельно записываться на приём, выбирая свободное время, и управлять своими записями; врач просматривает записи пациентов к себе и отмечает результат приёма; администратор имеет доступ ко всем записям с возможностью поиска и сортировки. Приложение обеспечивает проверку корректности вводимых данных, учёт занятых интервалов времени и разграничение прав доступа.

Практическая значимость работы заключается в том, что разработанное приложение может служить основой для организации электронной записи пациентов в медицинских учреждениях, а также использоваться в качестве учебного примера построения клиент-серверных веб-приложений с применением современного технологического стека и средств контейнеризации.

В качестве направлений дальнейшего развития приложения можно выделить внедрение защищённого хранения паролей с использованием хеширования и токенов доступа, добавление уведомлений пациентам о предстоящем приёме, расширение справочников за счёт нескольких городов и медицинских учреждений, а также организацию полноценного конвейера непрерывной интеграции и доставки (CI/CD) для автоматизации сборки и развёртывания приложения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) ГОСТ Р 7.0.100–2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – Москва : Стандартинформ, 2018. – 73 с.
- 2) ГОСТ 19.701–90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения. – Москва : Стандартинформ, 2010. – 26 с.
- 3) ГОСТ 34.601–90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания. – Москва : Издательство стандартов, 1992. – 12 с.
- 4) ГОСТ Р ИСО/МЭК 25010–2015. Информационные технологии. Системная и программная инженерия. (SQuaRE). – Москва : Стандартинформ, 2015. – 36 с.
- 5) ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. – Москва : Стандартинформ, 2017. – 27 с.
- 6) ГОСТ Р 50922–2019. Защита информации. Основные термины и определения. – Москва : Стандартинформ, 2019. – 12 с.
- 7) Федеральный закон от 27.07.2006 № 152-ФЗ (ред. от 24.06.2025) «О персональных данных». – Москва, 2025.
- 8) Федеральный закон от 27.07.2006 № 149-ФЗ (ред. от 29.12.2025) «Об информации, информационных технологиях и о защите информации». – Москва, 2025.
- 9) СанПиН 2.2.2/2.4.1340–03. Гигиенические требования к персональным электронно-вычислительным машинам и организации работы (с изменениями от 21.06.2016). – Москва, 2016.
- 10) ГОСТ Р 58771–2019. Менеджмент риска. Технологии оценки риска. – Москва : Стандартинформ, 2020. – 90 с.

- 11) Фримен, Э. Изучаем программирование на JavaScript / Э. Фримен, Э. Робсон. – Санкт-Петербург : Питер, 2019. – 640 с.
- 12) Бэнкс, А. React и Redux: функциональная веб-разработка / А. Бэнкс, Е. Порселло. – Санкт-Петербург : Питер, 2018. – 336 с.
- 13) Кантелон, М. Node.js в действии / А. Янг, Б. Мек, М. Кантелон [и др.]. – 2-е изд. – Санкт-Петербург : Питер, 2018. – 428 с.
- 14) Дакетт, Дж. HTML и CSS. Разработка и дизайн веб-сайтов / Дж. Дакетт. – Москва : Эксмо, 2017. – 480 с.
- 15) Бейли, Л. Изучаем SQL / Л. Бейли. – Санкт-Петербург : Питер, 2019. – 592 с.
- 16) Моуэт, Э. Использование Docker / Э. Моуэт. – Москва : ДМК Пресс, 2017. – 354 с.
- 17) Седер, Н. JavaScript для детей и взрослых. Самоучитель по программированию / Н. Седер. – Москва : Манн, Иванов и Фербер, 2019. – 336 с.
- 18) Node.js. Документация [Электронный ресурс]. – URL: <https://nodejs.org/docs> (дата обращения: 15.06.2026).
- 19) Express.js. Документация [Электронный ресурс]. – URL: <https://expressjs.com> (дата обращения: 15.06.2026).
- 20) MySQL. Документация [Электронный ресурс]. – URL: <https://dev.mysql.com/doc> (дата обращения: 15.06.2026).
- 21) Docker. Документация [Электронный ресурс]. – URL: <https://docs.docker.com> (дата обращения: 15.06.2026).
- 22) NGINX. Документация [Электронный ресурс]. – URL: <https://nginx.org/ru/docs> (дата обращения: 15.06.2026).
- 23) patientgo [Электронный ресурс]. – URL: <https://github.com/serzhoon/patientgo> (дата обращения: 20.06.2026).

ПРИЛОЖЕНИЕ А

Структура базы данных (schema.sql)

Листинг А.1 – Скрипт создания базы данных

```
-- =====
-- Схема базы данных для веб-приложения записи пациентов на приём
-- СУБД: MySQL 8.0
-- =====

SET NAMES utf8mb4;

CREATE DATABASE IF NOT EXISTS clinic
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE clinic;

-- -----
-- Таблица городов.
-- -----
CREATE TABLE IF NOT EXISTS cities (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  name    VARCHAR(100) NOT NULL
);

-- -----
-- Таблица поликлиник. Каждая привязана к городу.
-- -----
CREATE TABLE IF NOT EXISTS clinics (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  city_id     INT NOT NULL,
  name        VARCHAR(200) NOT NULL,
  CONSTRAINT fk_clinic_city FOREIGN KEY (city_id) REFERENCES cities(id)
  ON DELETE CASCADE
);

-- -----
-- Таблица врачей. Каждый врач привязан к поликлинике.
-- -----
CREATE TABLE IF NOT EXISTS doctors (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  clinic_id   INT NOT NULL,
  full_name   VARCHAR(150) NOT NULL,
  specialty   VARCHAR(150) NOT NULL,
  cabinet     VARCHAR(20),
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_doctor_clinic FOREIGN KEY (clinic_id) REFERENCES
  clinics(id) ON DELETE CASCADE
);

-- -----
-- Таблица пользователей (пациенты и администраторы).
-- Пациент привязан к поликлинике (clinic_id).
```

```

-----
CREATE TABLE IF NOT EXISTS users (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  last_name   VARCHAR(80)  NOT NULL,          -- фамилия
  first_name  VARCHAR(80)  NOT NULL,          -- имя
  middle_name VARCHAR(80),                    -- отчество
  (необязательно)
  birth_date  DATE,                          -- дата рождения (для
пациентов)
  email       VARCHAR(150) NOT NULL UNIQUE,
  phone       VARCHAR(30),
  password    VARCHAR(150) NOT NULL,          -- пароль (учебный
проект, хранится как текст)
  clinic_id   INT,                          -- к какой поликлинике
привязан пациент
  doctor_id   INT,                          -- если это аккаунт
врача — ссылка на его карточку
  role        ENUM('patient', 'admin', 'doctor') NOT NULL DEFAULT
'patient',
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_user_clinic FOREIGN KEY (clinic_id) REFERENCES
clinics(id) ON DELETE SET NULL,
  CONSTRAINT fk_user_doctor FOREIGN KEY (doctor_id) REFERENCES
doctors(id) ON DELETE CASCADE
);

```

```

-----
-- Таблица записей на приём.
-----

```

```

CREATE TABLE IF NOT EXISTS appointments (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  patient_id  INT NOT NULL,
  doctor_id   INT NOT NULL,
  apdate      DATE NOT NULL,
  apptime     TIME NOT NULL,
  status      ENUM('booked', 'cancelled', 'done', 'no_show') NOT NULL
DEFAULT 'booked',
  comment     VARCHAR(500),
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_patient FOREIGN KEY (patient_id) REFERENCES users(id)
ON DELETE CASCADE,
  CONSTRAINT fk_doctor FOREIGN KEY (doctor_id) REFERENCES doctors(id)
ON DELETE CASCADE
);

```

```

-----
-- Демонстрационные данные
-----

```

```

-- Город
INSERT INTO cities (id, name) VALUES (1, 'г. Ставрополь');

```

```

-- Поликлиники
INSERT INTO clinics (id, city_id, name) VALUES
  (1, 1, 'ГАУЗ СК "Городская поликлиника № 3"'),
  (2, 1, 'ГБУЗ СК "Городская клиническая поликлиника №6"'),

```

```

(3, 1, 'ГБУЗ СК "Ставропольский краевой клинический многопрофильный
центр"');

-- Врачи поликлиники №1 (ГБУЗ СК "Городская поликлиника № 3")
INSERT INTO doctors (clinic_id, full_name, specialty, cabinet) VALUES
(1, 'Иванова Анна Сергеевна', 'Терапевт', '101'),
(1, 'Петров Михаил Олегович', 'Хирург', '102'),
(1, 'Сидорова Елена Павловна', 'Кардиолог', '103'),
(1, 'Кузнецов Дмитрий Игоревич', 'Невролог', '104'),
(1, 'Морозова Ольга Викторовна', 'Офтальмолог', '105');

-- Врачи поликлиники №2 (ГБУЗ СК "Городская клиническая поликлиника №6")
INSERT INTO doctors (clinic_id, full_name, specialty, cabinet) VALUES
(2, 'Васильев Андрей Николаевич', 'Терапевт', '201'),
(2, 'Григорьева Мария Алексеевна', 'Эндокринолог', '202'),
(2, 'Соколов Павел Дмитриевич', 'Травматолог', '203'),
(2, 'Лебедева Татьяна Сергеевна', 'Дерматолог', '204'),
(2, 'Новиков Сергей Владимирович', 'Уролог', '205');

-- Врачи поликлиники №3 (Ставропольский краевой клинический
многопрофильный центр)
INSERT INTO doctors (clinic_id, full_name, specialty, cabinet) VALUES
(3, 'Фёдорова Ирина Анатольевна', 'Терапевт', '301'),
(3, 'Михайлов Виктор Петрович', 'Кардиохирург', '302'),
(3, 'Алексеева Наталья Игоревна', 'Гастроэнтеролог', '303'),
(3, 'Захаров Олег Геннадьевич', 'Онколог', '304'),
(3, 'Романова Светлана Юрьевна', 'Пульмонолог', '305');

-- Администратор регистратуры.
INSERT INTO users (last_name, first_name, middle_name, birth_date,
email, phone, password, clinic_id, role) VALUES
('Администратор', 'Регистратуры', NULL, NULL, 'admin@clinic.ru',
'+700000000000', 'Admin754', NULL, 'admin');

-- -----
-- Аккаунты врачей (заводит системный администратор).
-- doctor_id ссылается на карточку врача в таблице doctors (id 1..15
-- в порядке вставки выше).
-- -----
INSERT INTO users (last_name, first_name, middle_name, email, phone,
password, doctor_id, role) VALUES
-- Поликлиника №1
('Иванова', 'Анна', 'Сергеевна', 'ivanova@clinic.ru', NULL,
'Ivanova214', 1, 'doctor'),
('Петров', 'Михаил', 'Олегович', 'petrov@clinic.ru', NULL,
'Petrov125', 2, 'doctor'),
('Сидорова', 'Елена', 'Павловна', 'sidorova@clinic.ru', NULL,
'Sidorova859', 3, 'doctor'),
('Кузнецов', 'Дмитрий', 'Игоревич', 'kuznetsov@clinic.ru', NULL,
'Kuznetsov381', 4, 'doctor'),
('Морозова', 'Ольга', 'Викторовна', 'morozova@clinic.ru', NULL,
'Morozova350', 5, 'doctor'),
-- Поликлиника №2
('Васильев', 'Андрей', 'Николаевич', 'vasilev@clinic.ru', NULL,
'Vasilev328', 6, 'doctor'),
('Григорьева', 'Мария', 'Алексеевна', 'grigoreva@clinic.ru', NULL,
'Grigoreva242', 7, 'doctor'),

```

```
    ('Соколов', 'Павел', 'Дмитриевич', 'sokolov@clinic.ru', NULL,
'Sokolov854', 8, 'doctor'),
    ('Лебедева', 'Татьяна', 'Сергеевна', 'lebedeva@clinic.ru', NULL,
'Lebedeva204', 9, 'doctor'),
    ('Новиков', 'Сергей', 'Владимирович', 'novikov@clinic.ru', NULL,
'Novikov792', 10, 'doctor'),
    -- Поликлиника №3
    ('Фёдорова', 'Ирина', 'Анатольевна', 'fedorova@clinic.ru', NULL,
'Fedorova858', 11, 'doctor'),
    ('Михайлов', 'Виктор', 'Петрович', 'mihaylov@clinic.ru', NULL,
'Mihaylov658', 12, 'doctor'),
    ('Алексеева', 'Наталья', 'Игоревна', 'alekseeva@clinic.ru', NULL,
'Alekseeva189', 13, 'doctor'),
    ('Захаров', 'Олег', 'Геннадьевич', 'zaharov@clinic.ru', NULL,
'Zaharov704', 14, 'doctor'),
    ('Романова', 'Светлана', 'Юрьевна', 'romanova@clinic.ru', NULL,
'Romanova532', 15, 'doctor');
```

ПРИЛОЖЕНИЕ Б

Основные модули серверной части

Листинг Б.1 – Промежуточный обработчик проверки доступа (auth.js)

```
// =====
// Упрощённая проверка пользователя (без токенов).
// После входа фронтенд присылает в заголовках, кто он:
//   x-user-id    – номер пользователя
//   x-user-role  – роль (patient или admin)
// Здесь мы просто читаем эти заголовки и кладём в req.user.
// =====

// требует чтобы пользователь был залогинен (прислал id в
заголовке).
function authRequired(req, res, next) {
  const id = req.headers['x-user-id'];
  const role = req.headers['x-user-role'];
  const doctorId = req.headers['x-doctor-id']; // для роли "врач"
– его карточка

  if (!id) {
    return res.status(401).json({ error: 'Требуется вход' });
  }

  req.user = {
    id: Number(id),
    role: role || 'patient',
    doctor_id: doctorId ? Number(doctorId) : null
  };
  next();
}

// требует роль администратора
function adminRequired(req, res, next) {
  if (!req.user || req.user.role !== 'admin') {
    return res.status(403).json({ error: 'Доступ только для
администратора' });
  }
  next();
}

module.exports = { authRequired, adminRequired };
```

Листинг Б.2 – Контроллер аутентификации (authController.js)

```
// =====
// Контроллер аутентификации (упрощённый вариант).
// Регистрация пациента и вход. Пароль хранится как обычный текст
// и сравнивается напрямую – без хеширования и без токенов.
// =====

const pool = require('../config/db');
```

```

// --- Регистрация нового пациента ---
async function register(req, res) {
  try {
    const { last_name, first_name, middle_name, birth_date,
            email, phone, password, clinic_id } = req.body;

    // Проверка обязательных полей (отчество необязательно).
    if (!last_name || !first_name || !birth_date || !email ||
!password || !clinic_id) {
      return res.status(400).json({ error: 'Заполните все
обязательные поля' });
    }

    // Проверяем, нет ли уже пользователя с таким email.
    const [existing] = await pool.query('SELECT id FROM users WHERE
email = ?', [email]);
    if (existing.length > 0) {
      return res.status(409).json({ error: 'Пользователь с таким
email уже существует' });
    }

    const [result] = await pool.query(
      `INSERT INTO users (last_name, first_name, middle_name,
birth_date, email, phone, password, clinic_id, role)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`,
      [last_name, first_name, middle_name || null, birth_date,
email, phone || null, password, clinic_id, 'patient']
    );

    res.status(201).json({ message: 'Регистрация успешна', userId:
result.insertId });
  } catch (e) {
    console.error(e);
    res.status(500).json({ error: 'Ошибка сервера при регистрации'
});
  }
}

// --- Вход (логин) ---
async function login(req, res) {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ error: 'Введите email и
пароль' });
    }

    // Имя собираем из фамилии/имени/отчества. clinic_id нужен
пациенту для фильтра врачей,
    // doctor_id нужен врачу, чтобы показать записи к нему.
    const [rows] = await pool.query(
      `SELECT id,
      TRIM(CONCAT(last_name, ' ', first_name, ' ',
COALESCE(middle_name, ''))) AS full_name,
      email, role, clinic_id, doctor_id
      FROM users WHERE email = ? AND password = ?`,

```

```

        [email, password]
    );

    if (rows.length === 0) {
        return res.status(401).json({ error: 'Неверный email или
пароль' });
    }

    res.json({ user: rows[0] });
} catch (e) {
    console.error(e);
    res.status(500).json({ error: 'Ошибка сервера при входе' });
}
}

module.exports = { register, login };

```

Листинг Б.3 – Контроллер записей на приём (appointmentController.js)

```

// =====
// Контроллер записей на приём.
// Пациент: создаёт запись, смотрит и отменяет свои.
// Админ: видит все записи.
// =====

const pool = require('../config/db');

// Создать запись на приём (пациент).
// Стандартные слоты времени приёма (одинаковые для всех врачей).
const ALL_SLOTS =
['09:00', '09:30', '10:00', '10:30', '11:00', '11:30', '12:00',
'13:00', '13:30', '14:00', '14:30', '15:00'];

// Вернуть свободные слоты врача на конкретную дату (занятые
исключаются).
async function freeSlots(req, res) {
    try {
        const { doctor_id, appdate } = req.query;
        if (!doctor_id || !appdate) {
            return res.status(400).json({ error: 'Нужны врач и дата' });
        }

        // Занятые слоты этого врача на эту дату (активные записи).
        const [busy] = await pool.query(
            `SELECT apptime FROM appointments
            WHERE doctor_id = ? AND appdate = ? AND status = 'booked`,
            [doctor_id, appdate]
        );
        const busyTimes = busy.map(r => (r.apptime || '').slice(0, 5));

        // Оставляем только свободные.
        const free = ALL_SLOTS.filter(s => !busyTimes.includes(s));
        res.json(free);
    } catch (e) {
        console.error(e);
    }
}

```

```

        res.status(500).json({ error: 'Ошибка сервера' });
    }
}

async function createAppointment(req, res) {
    try {
        const { doctor_id, appdate, apptime, comment } = req.body;
        const patient_id = req.user.id; // берём из токена, а не из
тела запроса

        if (!doctor_id || !appdate || !apptime) {
            return res.status(400).json({ error: 'Выберите врача, дату и
время' });
        }

        // Проверяем, что слот у этого врача ещё свободен.
        const [busy] = await pool.query(
            `SELECT id FROM appointments
            WHERE doctor_id = ? AND appdate = ? AND apptime = ? AND
status = 'booked'`,
            [doctor_id, appdate, apptime]
        );
        if (busy.length > 0) {
            return res.status(409).json({ error: 'Это время уже занято,
выберите другое' });
        }

        const [result] = await pool.query(
            `INSERT INTO appointments (patient_id, doctor_id, appdate,
apptime, comment)
            VALUES (?, ?, ?, ?, ?)`,
            [patient_id, doctor_id, appdate, apptime, comment || null]
        );

        res.status(201).json({ message: 'Вы записаны на приём',
appointmentId: result.insertId });
    } catch (e) {
        console.error(e);
        res.status(500).json({ error: 'Ошибка сервера при создании
записи' });
    }
}

// Список записей.
// Пациент видит только свои; админ — все.
async function listAppointments(req, res) {
    try {
        let rows;
        if (req.user.role === 'admin') {
            [rows] = await pool.query(
                `SELECT a.*, d.full_name AS doctor_name, d.specialty,
                TRIM(CONCAT(u.last_name, ' ', u.first_name, ' ',
COALESCE(u.middle_name, ''))) AS patient_name,
                u.phone AS patient_phone
                FROM appointments a
                JOIN doctors d ON a.doctor_id = d.id
                JOIN users u   ON a.patient_id = u.id`
            );
        }
    }
}

```



```

        ORDER BY a.appdate, a.apptime`
    );
    } else if (req.user.role === 'doctor') {
        // Врач видит записи к себе (активные и завершённые, без
отменённых).
        [rows] = await pool.query(
            `SELECT a.*, d.full_name AS doctor_name, d.specialty,
                TRIM(CONCAT(u.last_name, ' ', u.first_name, ' ',
COALESCE(u.middle_name, ''))) AS patient_name,
                u.phone AS patient_phone
            FROM appointments a
            JOIN doctors d ON a.doctor_id = d.id
            JOIN users u ON a.patient_id = u.id
            WHERE a.doctor_id = ? AND a.status <> 'cancelled'
            ORDER BY a.appdate, a.apptime`,
            [req.user.doctor_id]
        );
    } else {
        [rows] = await pool.query(
            `SELECT a.*, d.full_name AS doctor_name, d.specialty
            FROM appointments a
            JOIN doctors d ON a.doctor_id = d.id
            WHERE a.patient_id = ? AND a.status NOT IN ('cancelled',
'no_show')
            ORDER BY a.appdate, a.apptime`,
            [req.user.id]
        );
    }
    res.json(rows);
} catch (e) {
    console.error(e);
    res.status(500).json({ error: 'Ошибка сервера' });
}
}

// Отменить запись.
// Пациент может отменить только свою; админ — любую.
async function cancelAppointment(req, res) {
    try {
        const { id } = req.params;

        // Сначала проверяем, чья это запись.
        const [rows] = await pool.query('SELECT * FROM appointments
WHERE id = ?', [id]);
        if (rows.length === 0) {
            return res.status(404).json({ error: 'Запись не найдена' });
        }

        const appt = rows[0];
        if (req.user.role !== 'admin' && appt.patient_id !==
req.user.id) {
            return res.status(403).json({ error: 'Можно отменять только
свои записи' });
        }

        await pool.query("UPDATE appointments SET status = 'cancelled'
WHERE id = ?", [id]);
    }
}

```

```

        res.json({ message: 'Запись отменена' });
    } catch (e) {
        console.error(e);
        res.status(500).json({ error: 'Ошибка сервера' });
    }
}

// Отметить приём состоявшимся (только врач, только к себе).
async function completeAppointment(req, res) {
    try {
        const { id } = req.params;

        if (req.user.role !== 'doctor') {
            return res.status(403).json({ error: 'Только врач может
отметить приём' });
        }

        const [rows] = await pool.query('SELECT * FROM appointments
WHERE id = ?', [id]);
        if (rows.length === 0) {
            return res.status(404).json({ error: 'Запись не найдена' });
        }

        // Врач может отмечать только записи к себе.
        if (rows[0].doctor_id !== req.user.doctor_id) {
            return res.status(403).json({ error: 'Можно отмечать только
записи к себе' });
        }

        await pool.query("UPDATE appointments SET status = 'done' WHERE
id = ?", [id]);
        res.json({ message: 'Приём отмечен как состоявшийся' });
    } catch (e) {
        console.error(e);
        res.status(500).json({ error: 'Ошибка сервера' });
    }
}

// Отметить неявку пациента (только врач, только к себе).
async function noShowAppointment(req, res) {
    try {
        const { id } = req.params;

        if (req.user.role !== 'doctor') {
            return res.status(403).json({ error: 'Только врач может
отметить неявку' });
        }

        const [rows] = await pool.query('SELECT * FROM appointments
WHERE id = ?', [id]);
        if (rows.length === 0) {
            return res.status(404).json({ error: 'Запись не найдена' });
        }

        if (rows[0].doctor_id !== req.user.doctor_id) {
            return res.status(403).json({ error: 'Можно отмечать только
записи к себе' });
        }
    }
}

```

```

    }

    await pool.query("UPDATE appointments SET status = 'no_show'
WHERE id = ?", [id]);
    res.json({ message: 'Отмечена неявка пациента' });
  } catch (e) {
    console.error(e);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
}

module.exports = { createAppointment, listAppointments,
cancelAppointment, completeAppointment, noShowAppointment, freeSlots };

```

ПРИЛОЖЕНИЕ В

Клиентская часть приложения

Листинг В.1 – Разметка интерфейса (index.html)

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>МедЗапись</title>
  <link rel="icon" type="image/svg+xml"
href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
viewBox='0 0 100 100'%3E%3Crect width='100' height='100' rx='20'
fill='%230e9488'/%3E%3Crect x='42' y='22' width='16' height='56' rx='3'
fill='%23fff'/%3E%3Crect x='22' y='42' width='56' height='16' rx='3'
fill='%23fff'/%3E%3C/svg%3E">
  <link rel="stylesheet" href="css/style.css">
</head>
<body>

  <header>
    <div class="brand">
      <svg class="brand-logo" viewBox="0 0 100 100" width="28"
height="28" xmlns="http://www.w3.org/2000/svg">
        <rect width="100" height="100" rx="20" fill="#0e9488"/>
        <rect x="42" y="22" width="16" height="56" rx="3"
fill="#fff"/>
        <rect x="22" y="42" width="56" height="16" rx="3"
fill="#fff"/>
      </svg>
      <span>МедЗапись</span>
    </div>
    <nav>
      <span id="userInfo"></span>
      <button id="logoutBtn" class="btn-logout
hidden">Выйти</button>
    </nav>
  </header>

  <div class="container">

    <!-- ===== Блок входа / регистрации (виден, пока не вошли)
===== -->
    <div id="authSection" class="card" style="max-width:420px;
margin:40px auto;">
      <div class="tabs">
        <button id="tabLogin" class="active">Вход</button>
        <button id="tabRegister">Регистрация</button>
      </div>

      <!-- Форма входа -->
      <div id="loginForm">
        <label for="loginEmail">Email</label>
```

```

        <input                                type="email"                                id="loginEmail"
placeholder="you@example.ru">
        <label for="loginPassword">Пароль</label>
        <input                                type="password"                                id="loginPassword"
placeholder=".....">
        <button class="btn-primary" id="loginBtn">Войти</button>
        <div                                class="forgot-link"><a                                href="#"
id="forgotLink">Забыли пароль?</a></div>
        <div class="divider">или</div>
        <button class="btn-gosuslugi" id="gosuslugiBtn">Войти
через Госуслуги</button>
        <div id="loginMsg" class="message"></div>
    </div>

    <!-- Форма регистрации -->
    <div id="registerForm" class="hidden">
        <label                                for="regLastName">Фамилия                                <span
class="req">*</span></label>
        <input type="text" id="regLastName" placeholder="Иванов">
        <label                                for="regFirstName">Имя                                <span
class="req">*</span></label>
        <input type="text" id="regFirstName" placeholder="Иван">
        <label for="regMiddleName">Отчество</label>
        <input                                type="text"                                id="regMiddleName"
placeholder="Иванович">
        <label for="regBirthDate">Дата рождения                                <span
class="req">*</span></label>
        <input type="date" id="regBirthDate">
        <label                                for="regCity">Город                                <span
class="req">*</span></label>
        <select id="regCity"></select>
        <label                                for="regClinic">Поликлиника                                <span
class="req">*</span></label>
        <select id="regClinic"></select>
        <label                                for="regEmail">Email                                <span
class="req">*</span></label>
        <input                                type="email"                                id="regEmail"
placeholder="you@example.ru">
        <label                                for="regPhone">Телефон                                <span
class="req">*</span></label>
        <input type="text" id="regPhone" placeholder="+7 (____)
____-____-____">
        <label                                for="regPassword">Пароль                                <span
class="req">*</span></label>
        <input                                type="password"                                id="regPassword"
placeholder="придумайте пароль">

        <label class="consent">
            <input type="checkbox" id="regConsent">
            <span>Я даю согласие на обработку персональных данных в
соответствии с требованиями №152-ФЗ от 27.07.2006.</span>
        </label>

        <p class="req-note"><span class="req">*</span> – поля,
обязательные для заполнения</p>
        <p class="mfc-note">Внимание: после регистрации изменить
персональные данные можно только при обращении в МФЦ.</p>

```

```

        <button                                class="btn-primary"
id="registerBtn">Зарегистрироваться</button>
        <div id="registerMsg" class="message"></div>
    </div>
</div>

<!-- ===== Блок приложения (виден после входа) ===== -->
<div id="appSection" class="hidden">

    <!-- Запись на приём – только для пациентов -->
    <div id="bookingCard" class="card">
        <h2>Записаться на приём</h2>
        <label for="doctorSelect">Врач</label>
        <select id="doctorSelect"></select>
        <label for="appdate">Дата</label>
        <input type="date" id="appdate">
        <label for="apptime">Время</label>
        <select id="apptime">
            <option value="">Сначала выберите врача и дату</option>
        </select>
        <label          for="comment">Жалоба          /          комментарий
(необязательно)</label>
        <textarea id="comment" placeholder="Например: головная
боль третий день"></textarea>
        <button                                class="btn-primary"
id="bookBtn">Записаться</button>
        <div id="bookMsg" class="message"></div>
    </div>

    <!-- Список записей -->
    <div class="card">
        <h2 id="apptTitle">Мои записи</h2>
        <div id="apptList"></div>
    </div>

</div>

</div>
<!-- Универсальное модальное окно (уведомления и подтверждения)
-->
<div id="uiModal" class="modal-overlay hidden">
    <div class="modal-window">
        <p          id="uiModalText"          style="margin-bottom:20px;font-
size:15px;"></p>
        <div          style="display:flex;gap:10px;justify-content:flex-
end;">
            <button                                class="btn-logout"
id="uiModalCancel">Отмена</button>
            <button          class="btn-primary"          id="uiModalOk"
style="width:auto;margin-top:0;">OK</button>
        </div>
    </div>
</div>
<!-- Модальное окно "Забыли пароль" -->
<div id="forgotModal" class="modal-overlay hidden">
    <div class="modal-window">

```

```

        <button                                class="modal-close"
id="forgotClose">&times;</button>
        <h2>Восстановление пароля</h2>
        <p    style="color:var(--muted);    font-size:14px;    margin-
bottom:8px;">
            Введите email, привязанный к аккаунту.
        </p>
        <label for="forgotInput">email</label>
        <input                                id="forgotInput"
placeholder="you@example.ru">
        <button                                class="btn-primary"
id="forgotSubmit">Восстановить</button>
        <div id="forgotMsg" class="message"></div>
    </div>
</div>
<!-- Подвал: контакты поддержки и копирайт -->
<footer class="site-footer">
    <div class="footer-support">
        <span>Поддержка:</span>
        <a href="tel:+78652550000">8 (8652) 55-00-00</a>
        <a href="tel:+79280000000">+7 (928) 000-00-00</a>
        <a href="mailto:support@clinic.ru">support@clinic.ru</a>
        <span class="footer-hours">Пн-Пт, 8:00-20:00</span>
    </div>
    <div class="footer-copy">© Гадиян Сергей, 2026</div>
</footer>
<script src="config.js"></script>
<script src="js/app.js"></script>
</body>
</html>

```